

Технічне завдання

IoT Device Network Log Intelligence Pipeline

Production-style Data Engineering and Cloud Engineering project with Go, Kafka, Python, PySpark, Airflow, Snowflake/BigQuery, dbt, Streamlit, Docker, Terraform and AWS

Параметр	Опис
Тип проекту	Data Engineering pipeline для обробки IoT network logs з локальним MVP та AWS cloud deployment
Рівень складності	Junior+ / Middle- для локальної реалізації; ближче до Middle-рівня при AWS deployment, IaC, PySpark, observability та CI/CD
Основна ціль	Перетворити raw device logs у чисті таблиці, SQL-метрики, anomaly alerts, data quality reports і dashboard
Основний стек	Go, Kafka, Python, PySpark, Airflow, Snowflake або BigQuery, dbt, SQL, Streamlit, Docker
Cloud / DevOps stack	AWS S3, Lambda, Step Functions, CloudWatch, Terraform, GitHub Actions, Git worktree workflow

1. Мета проекту

Потрібно розробити data engineering pipeline, який приймає сирі IoT network logs, обробляє їх, очищає від невалідних записів, зберігає у data warehouse, будує аналітичні таблиці та візуалізує ключові метрики для моніторингу, troubleshooting і виявлення аномалій.

Оновлена версія проекту також має показати cloud engineering підхід: інфраструктура описується через Terraform, raw/processed дані можуть зберігатися в AWS S3, окремі події обробляються через Lambda, cloud workflow координується Step Functions, а logs/metrics/alerts збираються в CloudWatch.

Проект демонструє навички ingestion pipeline, streaming, ETL/ELT, SQL-аналітики, data warehouse modeling, dbt transformations, orchestration через Airflow або Step Functions, dashboarding, data quality, observability, IaC та cloud deployment.

2. Що додано в оновлену версію

- IaC через Terraform для створення AWS ресурсів, warehouse ресурсів, IAM roles та environment-specific конфігурації.
- PySpark як advanced batch processing layer для великих обсягів історичних логів, дедуплікації, window aggregations і Parquet output.
- AWS deployment layer: S3, Lambda, Step Functions, CloudWatch, IAM, опціонально ECS/Fargate, Glue або EMR.
- Git worktree як developer workflow для паралельної роботи з гілками infra, dbt, dashboard, Airflow та AWS deployment.
- CI/CD через GitHub Actions: lint, tests, dbt compile/test, Terraform validate/plan, Docker build.

3. Короткий опис для GitHub

This project is a production-style IoT Log Intelligence Pipeline built with Go, Kafka, Python, PySpark, Airflow, Snowflake/BigQuery, dbt, Streamlit, Docker, Terraform and AWS. It simulates ingestion of IoT network logs, processes and validates records, stores raw and processed data in a cloud warehouse and/or S3 data lake, applies dbt transformations, calculates troubleshooting metrics, detects anomalies, and visualizes insights through an interactive dashboard. The advanced scope includes AWS deployment using S3, Lambda, Step Functions, CloudWatch and Terraform.

4. Dataset

Основний датасет: Kaggle IoT Device Network Logs або інший схожий набір IoT/network log data. Кожен запис розглядається як подія, згенерована IoT-пристроєм або network device.

Поле	Опис
event_timestamp	Час події або згенерований timestamp
device_id	Ідентифікатор пристрою. Якщо його немає в датасеті, його можна створити з IP/source-полів
source_ip / destination_ip	Адреси джерела та призначення
protocol	Network protocol, наприклад TCP/UDP/ICMP
packet_size / duration_ms	Технічні параметри події
event_type	normal / warning / error / attack
attack_type / status	Тип атаки, статус або label, якщо є в датасеті

5. Обов'язковий стек і роль кожного інструмента

Інструмент	Роль у проєкті
Go	Окремий ingestion service: читає raw logs, симулює stream і надсилає події в Kafka або інший streaming layer
Kafka	Message broker між ingestion layer та processing layer; локально через Docker, advanced варіант - AWS MSK або Kinesis
Python	Consumer, validation, cleaning, transformations, loading, anomaly detection helpers
PySpark	Advanced batch processing: heavy cleaning, deduplication, window aggregations, Parquet output, historical reprocessing
Airflow	Локальна або self-hosted orchestration: запуск ingestion, processing, warehouse load, dbt, data quality checks
AWS Step Functions	Cloud orchestration: state machine для S3 trigger, validation, processing, warehouse load, dbt step, quality checks і alerts
AWS S3	Data lake storage: bronze raw logs, silver cleaned logs, invalid records, Parquet outputs, reports, screenshots
AWS Lambda	Event-driven tasks: п'єрає на S3 upload, запускає Step Functions, виконує lightweight validation, пише metadata або alerts
AWS CloudWatch	Logs, metrics, alarms, pipeline health, Lambda logs, Step Functions execution status
Snowflake / BigQuery	Cloud data warehouse для raw, clean та analytics tables. Для AWS-only варіанту можна додати Redshift як optional alternative
dbt	SQL transformations, data modeling, tests, documentation, lineage
SQL	Аналітичні запити, aggregation, troubleshooting metrics
Streamlit	Dashboard для моніторингу, device breakdown, anomalies, data quality
Docker	Відтворюване локальне середовище для Kafka, Airflow, services і dashboard
Terraform	Infrastructure as Code: S3, IAM, Lambda, Step Functions, CloudWatch, warehouse resources, environments
Git worktree	Developer workflow для паралельної роботи з feature branches: infra, dbt, dashboard, Airflow, AWS deployment
GitHub Actions	CI/CD: lint, unit tests, dbt checks, Terraform checks, Docker build

6. Архітектура системи

6.1 Local MVP architecture

Kaggle IoT Dataset / Simulated Logs

- > Go Log Producer
- > Kafka Topic: iot_raw_logs
- > Python Consumer / Processing Layer
- > Kafka Topics: iot_processed_logs, iot_invalid_logs, iot_pipeline_alerts
- > Airflow Orchestration
- > Snowflake / BigQuery Raw Layer
- > dbt Transformations: Staging -> Intermediate -> Marts
- > SQL Metrics
- > Streamlit Dashboard
- > Alerts / Troubleshooting View

6.2 AWS Advanced architecture

```
Raw IoT logs
-> S3 Bronze bucket
-> Lambda trigger
-> Step Functions state machine
-> Go Producer / Python Processor / PySpark job
-> Kafka / MSK / Kinesis optional streaming layer
-> S3 Silver bucket and Warehouse staging tables
-> dbt Transformations: Staging -> Intermediate -> Marts
-> Streamlit Dashboard / ECS optional hosting
-> CloudWatch logs, metrics and alarms
```

6.3 Рекомендований підхід

Не потрібно одразу переносити всю логіку в AWS. Краще мати локальний MVP, який можна запустити через Docker Compose, і окремий advanced cloud deployment layer. Це покаже, що проєкт можна розробляти локально, тестувати, а потім переносити в cloud середовище через Terraform.

7. Kafka topics / Streaming events

Topic	Призначення
iot_raw_logs	Сирі події від Go producer
iot_invalid_logs	Невалідні записи або corrupted JSON
iot_processed_logs	Валідовані та нормалізовані події
iot_pipeline_alerts	Повідомлення про anomaly, spike або threshold alerts

Локально використовується Kafka в Docker Compose. В AWS advanced scope можна використати Amazon MSK або Kinesis, але це optional. Для портфоліо достатньо залишити Kafka локально і використати AWS для storage, orchestration та observability.

8. Data layers

8.1 Bronze layer - Raw data

Зберігає сирі дані майже без змін для auditability та можливості повторної обробки.

```
raw_iot_logs(  
    raw_id, ingestion_timestamp, source_file, kafka_offset,  
    raw_payload, is_valid_json, created_at  
)
```

S3 path example:

```
s3://iot-log-pipeline-dev/bronze/raw_iot_logs/year=YYYY/month=MM/day=DD/
```

8.2 Silver layer - Clean data

Містить очищені та нормалізовані події, окремі invalid records та довідник devices.

```
clean_network_events(  
    event_id, event_timestamp, device_id, source_ip, destination_ip,  
    protocol, packet_size, duration_ms, event_type, status,  
    attack_type, is_attack, created_at  
)  
invalid_logs(invalid_id, raw_id, error_reason, raw_payload, failed_at)  
devices(device_id, first_seen, last_seen, total_events, device_status)
```

S3 path example:

```
s3://iot-log-pipeline-dev/silver/clean_network_events/year=YYYY/month=MM/day=DD/
```

8.3 Gold layer - Analytics marts

Готові таблиці для dashboard, troubleshooting та аналітики.

```
mart_device_daily_metrics  
mart_protocol_hourly_metrics  
mart_security_events  
mart_anomaly_detection
```

mart_pipeline_quality

8.4 Operational metadata

```
pipeline_runs(  
  run_id, dag_id, execution_id, started_at, finished_at, status,  
  raw_records_count, processed_records_count, invalid_records_count,  
  duplicates_count, error_message  
)
```

9. Functional requirements

9.1 Go Producer

- Читає CSV/JSON logs з локального файлу, sample dataset або S3 object.
- Симулює надходження подій у near real-time режимі.
- Конвертує кожен запис у JSON message.
- Надсилає повідомлення в Kafka topic `iot_raw_logs`.
- Логуватиме кількість надісланих, пропущених і помилкових записів.

9.2 Python Consumer / Processing Layer

- Читає повідомлення з Kafka topic `iot_raw_logs`.
- Перевіряє формат JSON і обов'язкові поля.
- Обробляє missing fields, invalid timestamps, invalid protocol, negative values, duplicates.
- Валідні записи відправляє в `iot_processed_logs` і завантажує у warehouse або S3 Silver.
- Невалідні записи відправляє в `iot_invalid_logs` та зберігає `error_reason`.

9.3 PySpark Batch Processing - Advanced

- Читає raw logs з S3 Bronze або локальної data папки.
- Застосовує schema enforcement, type casting, timestamp parsing і standardization.
- Виконує deduplication по event_id або composite key: device_id + timestamp + source_ip + destination_ip.
- Рахує hourly/daily aggregations через window functions.
- Пише cleaned output у partitioned Parquet в S3 Silver або локальну папку.
- Запускається через Airflow SparkSubmitOperator, Docker job, AWS Glue або EMR у advanced cloud scope.

9.4 AWS Lambda - Advanced

- Запускається при появі нового файлу в S3 Bronze bucket.
- Перевіряє базову структуру файлу: extension, size, empty file, naming convention.
- Стартує Step Functions workflow для подальшої обробки.
- Пише lightweight metadata у pipeline_runs або CloudWatch logs.
- Може надсилати alert, якщо файл некоректний або workflow не стартував.

9.5 AWS Step Functions - Advanced

- Координує cloud workflow як state machine.
- Має кроки: check input -> validate -> process -> load -> dbt -> quality checks -> notify.
- Підтримує retry/catch logic для fail-safe обробки.
- Записує execution status у CloudWatch і pipeline metadata.

9.6 Terraform IaC

- Описує S3 buckets для bronze/silver/gold/reports.

- Описує IAM roles/policies для Lambda, Step Functions, CloudWatch і доступу до S3.
- Описує Lambda function, Step Functions state machine, CloudWatch log groups і alarms.
- Має environments: dev та prod або local та cloud.
- Має terraform validate, terraform fmt і terraform plan у CI/CD.

10. Data validation requirements

Перевірка	Приклад помилки
Required fields	missing_device_id, missing_timestamp
Timestamp parsing	invalid_timestamp
Protocol validation	invalid_protocol
Numerical checks	negative_packet_size, invalid_duration
Duplicate detection	duplicate_event
JSON parsing	invalid_json
Schema drift	unknown_column, missing_expected_column
Freshness	source_file_is_too_old, no_new_records
Row count checks	processed_count_lower_than_expected

11. dbt requirements

Шар dbt	Приклади моделей	Задача
Staging	stg_iot_raw_logs stg_iot_processed_logs stg_iot_invalid_logs	Стандартизація назв колонок, типів даних і timestamp
Intermediate	int_clean_network_events int_device_activity int_security_events	Підготовка clean events, device activity, security-related events
Marts	mart_device_daily_metrics mart_attack_trends mart_anomaly_detection mart_pipeline_quality	Фінальні aggregation tables для dashboard

dbt також має містити tests: not_null, unique, accepted_values, relationships, freshness checks, row count checks та документацію через dbt docs.

12. SQL metrics

- Event volume over time: кількість подій по годинах або днях.
- Error / attack rate per device: частка abnormal events для кожного device_id.
- Most frequent attack types: найчастіші типи атак або suspicious events.
- Top risky devices: пристрої з найбільшою кількістю abnormal events.
- Protocol breakdown: розподіл подій по network protocol.
- Active / inactive devices: пристрої, які давно не надсилали логи.
- MTBE: mean time between errors для кожного device_id.
- Spike detection: різкий ріст errors / attacks у певний період.
- Pipeline quality: processed vs invalid vs duplicate records по кожному запуску.
- Cloud monitoring: кількість failed Step Functions executions, Lambda errors, S3 files processed.

13. Anomaly detection

Мінімальний варіант anomaly detection: якщо кількість attack/error events за годину більша за середнє значення + 2 стандартні відхилення, період позначається як anomaly.

```
mart_anomaly_detection(  
    anomaly_id, detected_at, metric_name, device_id,  
    actual_value, expected_value, threshold, severity, description  
)
```

Advanced варіант: PySpark рахує rolling statistics по device_id/protocol, а dbt формує final anomaly mart для dashboard.

14. Dashboard requirements

Сторінка	Що показує
Overview	Total events, total devices, attacks/errors, attack rate, failed records, last pipeline status
Time-series	Event volume, attack/error volume, anomaly markers
Device monitoring	Top risky devices, device-level breakdown, active/inactive devices
Security / troubleshooting	Top attack types, suspicious spikes, affected devices, recent critical events
Data quality	Processed records, failed records, validation errors, duplicates, pipeline run history
Cloud health - Advanced	Step Functions status, Lambda errors, S3 files processed, CloudWatch alarms

15. Orchestration requirements

15.1 Airflow DAG requirements - Local / self-hosted

DAG	Tasks
iot_ingestion_dag	check_source_files, start_go_producer, check_kafka_topic, log_ingestion_status
iot_processing_dag	consume_raw_logs, validate_records, deduplicate_records, write_processed_logs, write_invalid_logs
iot_spark_processing_dag	submit_spark_cleaning_job, write_clean_parquet, load_spark_output_to_warehouse, validate_spark_output
iot_warehouse_load_dag	load_raw_to_warehouse, load_processed_to_warehouse, load_invalid_to_warehouse, verify_row_counts
iot_dbt_transformations_dag	dbt_seed, dbt_run_staging, dbt_run_intermediate, dbt_run_marts, dbt_test, dbt_docs_generate
iot_data_quality_dag	check_null_device_ids, check_invalid_timestamps, check_duplicate_events, check_failed_records_rate, create_quality_report

15.2 Step Functions workflow - AWS Advanced

```
S3 ObjectCreated event  
-> Lambda: validate_new_file  
-> Step Functions:  
    1. check_input_file  
    2. run_processing_job  
    3. write_to_s3_silver  
    4. load_to_warehouse  
    5. run_dbt_transformations  
    6. run_data_quality_checks  
    7. publish_success_or_failure_alert  
-> CloudWatch logs and alarms
```

16. Observability requirements

Pipeline має зберігати інформацію про кожен запуск у таблиці `pipeline_runs` та показувати стан системи у dashboard.

```
pipeline_runs(  
    run_id, dag_id, execution_id, started_at, finished_at, status,  
    raw_records_count, processed_records_count, invalid_records_count,  
    duplicates_count, error_message  
)
```

- Логувати start/end кожного етапу.
- Зберігати кількість raw, processed, invalid і duplicate records.
- Зберігати статус dbt tests і warehouse load.
- Показувати pipeline health у dashboard.
- В AWS advanced scope відправляти Lambda logs, Step Functions execution logs і custom metrics у CloudWatch.
- Створити CloudWatch alarms для Lambda errors, failed state machine executions, high invalid record rate, no new files in S3.

17. Docker requirements

Потрібно додати `docker-compose.yml`, який піднімає локальне середовище:

- Kafka
- Airflow webserver
- Airflow scheduler
- Python processing service
- Go producer service
- Streamlit dashboard
- Optional: Postgres for Airflow metadata, DuckDB local fallback
- Optional: Spark standalone container або локальний PySpark job для advanced batch processing

18. AWS Deployment requirements - Advanced

AWS сервіс	Що реалізувати
S3	Buckets або prefixes для bronze, silver, gold, invalid, reports, artifacts
Lambda	S3 trigger, lightweight validation, Step Functions starter, alert helper
Step Functions	State machine для cloud orchestration: validate -> process -> load -> dbt -> quality -> notify
CloudWatch	Log groups, metrics, alarms, execution monitoring
IAM	Least-privilege roles for Lambda, Step Functions, S3 access, logs access
ECS/Fargate - optional	Hosting for Go producer, Python consumer або Streamlit dashboard
Glue/EMR - optional	Managed PySpark execution для heavy batch processing

19. Infrastructure as Code requirements

Terraform має бути окремим шаром проєкту, а не просто набором інструкцій у README.

```
infra/terraform/  
  main.tf  
  variables.tf  
  outputs.tf  
  providers.tf  
  modules/  
    s3/  
    iam/  
    lambda/  
    step_functions/  
    cloudwatch/  
  environments/  
    dev/  
    prod/
```

- Для MVP можна реалізувати тільки dev environment.
- Не зберігати AWS keys у репозиторії.
- Додати .env.example і README з поясненням required variables.
- Додати terraform fmt, validate і plan у GitHub Actions.
- У README показати приклад terraform plan screenshot або output summary.

20. Git workflow requirements

Git worktree не є частиною runtime architecture. Це developer workflow, який показує, що з великим проєктом можна працювати структуровано.

```
git worktree add ../iot-pipeline-infra feature/terraform-infra  
git worktree add ../iot-pipeline-dbt feature/dbt-models  
git worktree add ../iot-pipeline-dashboard feature/streamlit-dashboard  
git worktree add ../iot-pipeline-aws feature/aws-deployment
```

- main - стабільна версія проєкту.
- develop - інтеграційна гілка.
- feature/terraform-infra - IaC зміни.
- feature/spark-processing - PySpark batch layer.
- feature/aws-deployment - Lambda, Step Functions, CloudWatch.
- feature/dbt-models - dbt transformations і tests.
- feature/dashboard - Streamlit dashboard.

21. CI/CD requirements

Pipeline	Checks
Go CI	go fmt, go test, static checks
Python CI	ruff/flake8, pytest, type checks optional
PySpark CI	unit tests for transformations on sample data
dbt CI	dbt deps, dbt compile, dbt test on test profile
Terraform CI	terraform fmt, terraform validate, terraform plan
Docker CI	docker build for producer, consumer, dashboard

22. Repository structure

```
iot-log-intelligence-pipeline/  
  README.md  
  docker-compose.yml  
  .env.example  
  data/  
  go-producer/  
  python-consumer/  
  spark/  
  airflow/dags/  
  dbt/models/staging/  
  dbt/models/intermediate/  
  dbt/models/marts/  
  sql/  
  dashboard/  
  aws/lambda/  
  aws/step-functions/  
  infra/terraform/  
  .github/workflows/  
  docs/screenshots/  
  docs/development-workflow.md  
  tests/
```

23. README requirements

- Project overview і короткий опис мети проєкту.
- Architecture overview: Local MVP і AWS Advanced architecture.
- Tooling decisions: чому обрано Go, Kafka, Airflow, dbt, warehouse, Streamlit, PySpark, Terraform і AWS.
- How to run locally: docker compose, Airflow DAG trigger, dbt run, dashboard access.
- How to deploy AWS advanced scope: Terraform init/plan/apply, required variables, S3 upload, Step Functions execution.
- Sample data description і посилання на повний датасет.
- Dashboard screenshots.
- CloudWatch screenshots або приклади logs/alarms.
- Example insights discovered from the data.
- Cost control note: як зупинити ресурси і не залишити активні сервіси.

24. Definition of Done

- Працює Go producer.
- Створені Kafka topics.
- Працює Python consumer / processor.
- Raw і processed дані завантажуються у Snowflake або BigQuery.
- Є dbt models і dbt tests.
- Є Airflow DAG-и для основних етапів pipeline.
- Є SQL-метрики для monitoring і troubleshooting.
- Є Streamlit dashboard.
- Є data quality tracking і pipeline observability.
- Є Docker Compose для локального запуску.
- Є Terraform IaC для AWS advanced layer або хоча б для частини інфраструктури.
- Є S3 Bronze/Silver/Gold або reports structure.
- Є Lambda trigger і Step Functions workflow у advanced scope.

- Є CloudWatch logs/metrics/alarms у advanced scope.
- Є PySpark batch processing job у advanced scope.
- Є README, screenshots, приклади insights і development workflow documentation.

25. MVP vs Advanced scope

Scope	Склад
MVP	Go producer, Kafka, Python consumer, Snowflake/BigQuery load, dbt staging + marts, Airflow DAG, Streamlit dashboard, Docker Compose, README
Advanced	Incremental loading, anomaly detection, dbt tests, pipeline_runs table, data quality dashboard, alert simulation, partitioning, CI/CD
Cloud Advanced	Terraform IaC, AWS S3, Lambda trigger, Step Functions workflow, CloudWatch logs/alarms, IAM roles, optional ECS/Fargate
Big Data Advanced	PySpark batch processing, partitioned Parquet, historical reprocessing, optional Glue/EMR
Developer Workflow	Git worktree, feature branches, GitHub Actions, docs/development-workflow.md

26. Recommended implementation phases

Фаза	Що зробити
Phase 1 - Local MVP	Docker Compose, Kafka, Go producer, Python consumer, warehouse load, dbt, dashboard
Phase 2 - Data quality	Validation rules, invalid logs, dbt tests, pipeline_runs, data quality dashboard
Phase 3 - IaC + S3	Terraform project, S3 buckets/prefixes, IAM roles, S3 upload flow
Phase 4 - AWS orchestration	Lambda trigger, Step Functions workflow, CloudWatch logs and alarms
Phase 5 - PySpark	Batch processing for large files, Parquet output, historical reprocessing
Phase 6 - CI/CD + polish	GitHub Actions, screenshots, architecture diagram, final README

27. Final note

Найкраща стратегія - не намагатися реалізувати все одночасно. Для GitHub важливо мати стабільний локальний MVP, а AWS, Terraform, PySpark і CloudWatch оформити як advanced cloud scope. Якщо реалізувати хоча б частину AWS deployment з хорошою документацією і скріншотами, проєкт буде виглядати значно сильніше за типовий junior portfolio project.